Module 6 — Claude Agent SDK

Lesson 6.2

Your first headless agent

~50 lines of Python and you have an agent that reads files, edits them, runs shell, and reports back.

Why it matters

The fastest way to internalize the SDK is to read and run a minimal example, then change it. This lesson walks through examples/05-sdk-agent-python.

What the agent does

- Takes a prompt from the command line
- Operates inside a sandbox working directory (so it can't touch your real files)
- Has Read, Edit, Glob, Grep, Bash tools available
- Streams events to stdout: assistant text, tool calls, tool results, final summary
- Exits with a non-zero code if the task wasn't completed

Run it:

```
cd examples/05-sdk-agent-python
pip install -e .
export ANTHROPIC_API_KEY=sk-...

python agent.py "create a hello world Python script and run it"
```

The walkthrough

The whole agent in essence:

```
import anyio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main(task: str, workdir: str):
    options = ClaudeAgentOptions(
        system_prompt="You are a careful, terse coding agent.",
        cwd=workdir,
        allowed_tools=["Read", "Write", "Edit", "Glob", "Grep", "Bash"],
        permission_mode="acceptEdits",
        max_turns=20,
    )

    async for event in query(prompt=task, options=options):
        # event is a message describing what happened: assistant text, tool call, etc.
        handle(event)

anyio.run(main, "create a hello.py and run it", "./sandbox")
```

What's happening:

1. `ClaudeAgentOptions` sets the system prompt, working dir, allowed tools, permissions, and a turn cap.
2. `query(prompt=..., options=...)` is the main entry point — it returns an async iterator of events.
3. Each event represents one step of the loop: a model message, a tool call, a tool result, a usage update.
4. The loop continues until the task is done or `max_turns` is hit.

`handle(event)` is yours to write — for our example, we print structured output. For a real service, you'd log events, persist state, push to a queue, etc.

What's in the example file

examples/05-sdk-agent-python/agent.py adds:

- CLI parsing (the task and the sandbox path come from argv)
- Sandbox creation (mkdir if missing)
- Pretty-printing of events
- A final summary block (tokens used, tools called, success/fail)

It's a small wrapper on top of the SDK call. The interesting part is the options.

Key options to know

```
ClaudeAgentOptions(
    model="claude-opus-4-7",              # default model
    system_prompt="...",                  # appended/replaces default
    cwd="/some/dir",                      # working directory for tools
    allowed_tools=[...],                  # subset of built-in tools
    disallowed_tools=[...],               # blacklist alternative
    permission_mode="acceptEdits",        # default | acceptEdits | bypassPermissions | plan
    max_turns=20,                         # safety cap
    mcp_servers={...},                    # optional MCP servers, same shape as Claude Code
    can_use_tool=callback,                # optional fine-grained permission gate
)
```

(Exact keys/names may evolve. Check the SDK README for your version.)

The `can_use_tool` callback is powerful — it's called for each tool invocation and can approve, deny, or modify. It's how you implement custom policies (e.g., "never write to anything outside `/sandbox`").

Working with events

The event stream is structured. Common event types:

- **AssistantMessage** — text from the model
- **ToolUseBlock** — the model wants to call a tool
- **ToolResultBlock** — the tool returned (or errored)
- **UserMessage** — system-injected (e.g., tool results being fed back)
- **ResultMessage** — final summary including usage and stop reason

You can:

- Print them for visibility
- Save them to a JSONL log
- Stream just the text content to stdout
- Filter for tool usage to build per-tool metrics

The sandbox pattern

For dev and untrusted task generation, always run in a sandbox:

```
sandbox = Path("./sandbox").resolve()
sandbox.mkdir(exist_ok=True)
options.cwd = str(sandbox)
```

Combined with `allowed_tools` that don't include destructive Bash, this gives you a reasonable isolation boundary on a dev machine. For real isolation (untrusted prompts, multi-tenant), use a container.

Try it

1. Run the example with `python agent.py "make a script that prints today's date"`.
2. Watch the event stream. Identify the tool calls.
3. Re-run with a more ambitious task: `python agent.py "write a CLI that takes a URL and prints the page title, with tests"`.
4. Change the system prompt to something terse: `"Output minimal commentary. Just do the task."`. Re-run, observe.

Gotchas

- **`anyio.run` vs. `asyncio.run`** — the SDK is async; pick a runner you like. `anyio` is portable across asyncio/trio.
- **The sandbox must exist** before passing as `cwd`. Pre-create.
- **`permission_mode="bypassPermissions"`** is the SDK's equivalent of `--dangerously-skip-permissions`. Use only in sandboxes.
- **`max_turns` is a hard cap.** If the agent hits it before completing, the task is incomplete — handle that in your wrapper.

Further reading

- examples/05-sdk-agent-python — full runnable
- 6.3 Tool use patterns — defining your own tools
- SDK docs: https://docs.claude.com/en/docs/agents/agent-sdk

Next

→ 6.3 Tool use patterns